

# 《红楼梦》前 80 回与后 40 回 写作风格差异分析 ——基于非参数统计方法

统计 2301 林煜翔 2233514230

2026 年 5 月 24 日

目录

1 背景介绍与研究问题 1

2 模型介绍 1

2.1 Wilcoxon 秩和检验 1

2.2 K-S 检验 2

2.3 置换检验 2

3 数据收集与预处理 3

3.1 数据收集 3

3.2 数据预处理 4

4 实验结果与分析 10

4.1 全特征 Mann-Whitney  $U$  检验结果 10

4.1.1 多重比较校正下的整体显著性 10

4.1.2 Top 10 差异词语及效应量 11

4.1.3 代表性差异词语的分组分布 11

4.2 PCA 降维与 PC1 补充检验 13

4.2.1 主成分方差解释 13

4.2.2 三维散点图的分组聚类趋势 14

4.2.3 PC1 的三项非参数检验 15

4.3 综合分析与讨论 16

A 实验代码 17

A.1 数据获取 17

A.2 数据预处理与可视化 18

A.3 全特征非参数检验 23

A.4 主成分分析、补充检验与剩余代码 25

# 1 背景介绍与研究问题

《红楼梦》是中国古典小说的巅峰之作，全书共 120 回，以贾宝玉、林黛玉、薛宝钗等人物命运为主线，展现了清代贵族家庭的兴衰图景。传统观点认为前 80 回为曹雪芹所著，后 40 回由高鹗续成，但学界对此长期存在争议：部分研究者认为后四十回仍出自曹雪芹之手，亦有观点主张续作者另有其人。由于作者身份难以直接考证，从文本本身的用词习惯、语言风格等可量化特征入手，检验前后两部分是否存在系统性差异，成为红学研究中一种可操作的实证路径。

文本风格分析依赖对大规模语料的预处理与特征提取。本研究以各回为样本单元，经分词、停用词过滤及人名剔除后，构建每章  $V = 2000$  维的高频非人名词词频特征矩阵，并对词频按列归一化以减弱绝对频次差异的影响。前 80 回与后 40 回构成两组独立样本，比较对象为各词语在各回中的相对使用强度。此类高维、分布形态未知且未必满足正态性的计数型数据，不宜简单套用参数检验；非参数方法——如 Mann-Whitney  $U$  检验、Kolmogorov-Smirnov 两样本检验及置换检验不要求总体分布假定，且配合 Bonferroni 与 FDR 多重比较校正，可在控制假阳性率的前提下识别前后两部分用词差异显著的词语。此外，主成分分析将高维词频投影至低维空间，便于从整体上观察 120 回的分组聚类趋势，并对解释方差最大的第一主成分做补充非参数检验，从全局与局部两个层面验证风格差异的统计证据。

为此，本研究以《红楼梦》全文为基础，构建从数据采集、文本预处理到非参数统计推断的完整分析流程：首先从公开网站抓取 120 回正文并本地存储；继而完成分词、特征构建、逐词 Mann-Whitney 检验与多重比较校正、PCA 降维可视化及 PC1 补充检验，并输出统计报告与图表。研究旨在回答如下问题：**前 80 回与后 40 回在用词风格上是否存在显著差异？**若差异词语在严格校正后仍显著，则与「后四十回作者存疑」的假设相容；若各项检验均未拒绝「前后风格相同」的零假设，则不支持基于用词特征的续作者差异论断。本研究还展示了非参数统计方法在古典文学文本计量分析中的适用性。

**数据来源：**<http://www.gudianmingzhu.com/guji/honglouloumeng/>。该站点为古典名著公开阅读平台，提供《红楼梦》按回目划分的完整章节列表与正文页面。

## 2 模型介绍

### 2.1 Wilcoxon 秩和检验

用于比较两个独立样本分布位置是否有显著差异。进行检验时，先将两组连续型数据混合并统一从小到大排序。在零假设成立的条件下，两组数据的秩次应呈现均匀、交叉的分布形式，各自的秩和应符

合理比例，并且构造零分布，进行检验。

不妨设两组样本量分别为  $n_1$  和  $n_2$ 。混合编秩后，第一组的秩和为  $R_1$ 。

$$U_1 = R_1 - \frac{n_1(n_1 + 1)}{2}$$

由于  $U_1 + U_2 = n_1 n_2$ ，最终检验统计量取两者的较小值  $U = \min(U_1, U_2)$ 。在大样本情况下，可利用渐进正态性进行假设检验：

$$Z = \frac{U - \frac{n_1 n_2}{2}}{\sqrt{\frac{n_1 n_2 (n_1 + n_2 + 1)}{12}}}$$

## 2.2 K-S 检验

用于检验两个独立连续型样本是否来自完全相同的总体分布。特别地，K-S 检验对两组分布的形状、位置和尺度（如方差、偏度等）差异十分敏感。统计量为两组样本经验累积分布函数（ECDF）之间的最大距离。

不妨设两组样本的经验累积分布函数分别为  $F_{1,n_1}(x)$  和  $F_{2,n_2}(x)$ ，则 K-S 检验统计量  $D$  为：

$$D = \sup_x |F_{1,n_1}(x) - F_{2,n_2}(x)|$$

## 2.3 置换检验

其基本零假设为分组标签与观测到的响应变量之间相互独立。注意到置换检验为一种基于重采样、且不依赖任何渐进分布假设的精确非参数检验方法。在零假设下，通过将所有样本的组别标签进行随机置换，可构建出检验统计量的零分布，并进一步计算检验  $p$  值。

设原始观测样本计算出的真实检验统计量为  $T_{\text{obs}}$ 。经过  $B$  次随机标签置换后，获得置换统计量序列  $\{T^{(b)}\}_{b=1}^B$ 。则双侧经验  $p$  值的计算公式为：

$$p = \frac{\sum_{b=1}^B \mathbb{I}(|T^{(b)}| \geq |T_{\text{obs}}|)}{B}$$

其中  $\mathbb{I}(\cdot)$  为示性函数。

## 3 数据收集与预处理

### 3.1 数据收集

本研究使用的《红楼梦》全文文本通过 Python 脚本 `DataCollection.py` 从网络公开资源自动采集获得。数据源为古典名著网站 <http://www.gudianmingzhu.com/guji/honglouloumeng/>，该站点提供按回目划分的完整章节列表及正文页面。采集流程如下。

#### 环境与存储

脚本运行前检查项目根目录下是否存在 红楼梦文件夹；若不存在则自动创建，用于存放各回正文。全书共 120 回，每回对应一个独立的 UTF-8 编码文本文件。

#### 索引页解析

首先向上述索引页发送 HTTP GET 请求。为降低被服务器拒绝访问的风险，请求头中设置浏览器 User-Agent 进行伪装。响应体编码根据 `apparent_encoding` 自动识别后解码为文本，再使用 BeautifulSoup 解析 HTML。

在索引页中通过 CSS 选择器 `#leftdg > div.dgcon.ui > div > a` 定位全部章节链接，分别提取：

- 各回在目录中的标题文本 (`titleList`)；
- 各回正文页面的相对或绝对 URL (`urlList`)。

由此得到 120 个章节标题与链接的一一对应关系。

#### 逐回正文抓取

对第 1 至 120 回依次循环处理。对每一回：

1. 根据 `urlList` 中对应链接再次发起 GET 请求，同样进行编码识别与文本解码；
2. 解析章节页面，用选择器 `#leftdg > div:nth-child(1) > div > div` 定位正文容器，汇总其下所有 `<p>` 标签中的纯文本，拼接为章节正文 `content`；

3. 从正文首段提取回目标题 `titleText`，并去除其中的全角空格3000；
4. 将目录标题与页面内回目标题合并，得到完整文件名所用标题；
5. 简单规范化：用正则表达式将连续两个全角空格3000{2} 替换为换行符，以保留段落结构。

## 本地保存

规范化后的正文写入本地文件，命名规则为 红楼梦/{回次序号}\_{完整标题}.txt。例如第 1 回保存为 1\_ 第一回甄士隐梦幻识通灵贾雨村风尘怀闺秀.txt。若同名文件已存在，脚本会先删除再重写，以保证内容与最新抓取结果一致。全部 120 回处理完毕后，红楼梦目录即构成后续清洗、分词与统计分析所需的原始语料库。

## 3.2 数据预处理

### 中文分词与文本过滤

对红楼梦/目录下 120 个章节文件逐章读取 UTF-8 编码正文,采用 jieba 精确模式(`cut_all=False`)进行中文分词。分词前从 `userdict.json` 加载人物别名表，共注册 255 个别名并标记为人名 (`nr`)，以提高人物名称识别准确率。同时加载 `stopwords.txt` 停用词表，过滤单字词（长度  $\leq 1$ ）及停用词，保留长度  $\geq 2$  的有效词。经上述处理后，全书累计有效词次 177889 个，不同词数（词汇量）39513 个。

### 人名剔除与特征词筛选

为避免人物专名主导风格特征、掩盖通用用词差异，在全书词频降序排列的基础上，利用正则匹配剔除含人物别名的高频词，去人名后保留 39018 个不同词。从中选取出现频次最高的  $V = 2000$  个非人名词作为后续分析的特征维度，构成”词袋”特征集合。

### 特征矩阵构建与归一化

以 120 回为行、2000 个特征词为列，构建章节—词语词频矩阵  $\mathbf{X} \in \mathbb{R}^{120 \times 2000}$ ，矩阵元素  $x_{ij}$  表示第  $i$  回中第  $j$  个特征词的出现次数。为消除不同词语绝对频次量级的差异，对每一列（词语维度）按该

词在 120 回中的最大频次进行归一化：

$$\tilde{x}_{ij} = \frac{x_{ij}}{\max_k x_{kj}}, \quad \tilde{x}_{ij} \in [0, 1]. \quad (1)$$

归一化后特征矩阵的非零元素占比约为 18.75%，呈现较高稀疏性。按传统版本划分，前 80 回 ( $n_1 = 80$ ) 作为对照组，后 40 回 ( $n_2 = 40$ ) 作为比较组，后续推断均基于此分组。

## 预处理结果及可视化

表 1 汇总了预处理后语料库的基本规模，表 2 给出各章三项基本指标的描述统计，表 3 对比前后两部分的关键差异，表 4 列出全书 Top 10 高频非人名词。

表 1: 预处理后语料库基本概况

| 统计项         | 数值      |
|-------------|---------|
| 章节总数        | 120     |
| 全书有效词次总数    | 177 889 |
| 全书不同词数（词汇量） | 39 513  |
| 去人名后不同词数    | 39 018  |
| 特征维度 $V$    | 2 000   |
| 特征矩阵非零占比    | 18.75%  |

表 2: 各章基本指标描述统计 ( $n = 120$ )

| 指标   | 均值       | 标准差    | 中位数      | 最小值 | $Q_1$    | $Q_3$    | 最大值   |
|------|----------|--------|----------|-----|----------|----------|-------|
| 有效词数 | 1 482.41 | 326.60 | 1 398.00 | 890 | 1 279.75 | 1 647.25 | 2 421 |
| 不同词数 | 940.01   | 219.36 | 907.00   | 574 | 795.00   | 1 009.25 | 1 771 |
| 段落数  | 24.47    | 15.13  | 19.00    | 10  | 15.75    | 27.00    | 85    |

表 3: 前 80 回与后 40 回基本指标对比

| 指标   | 分组     | 均值       | 标准差    | 中位数      | 极差        |
|------|--------|----------|--------|----------|-----------|
| 有效词数 | 前 80 回 | 1 516.56 | 355.07 | 1 458.50 | 890–2 421 |
|      | 后 40 回 | 1 414.10 | 250.85 | 1 362.50 | 978–2 132 |
| 不同词数 | 前 80 回 | 982.27   | 246.00 | 954.00   | 574–1 771 |
|      | 后 40 回 | 855.48   | 115.08 | 842.50   | 661–1 216 |

注：有效词数指经分词、去单字、去停用词后的词次总数；不同词数为各章类型数。

表 4: 全书 Top 10 高频非人名词

| 排名 | 词语 | 频次  | 排名 | 词语 | 频次  |
|----|----|-----|----|----|-----|
| 1  | 说道 | 938 | 6  | 奶奶 | 706 |
| 2  | 众人 | 816 | 7  | 听见 | 657 |
| 3  | 姑娘 | 755 | 8  | 不知 | 647 |
| 4  | 只见 | 746 | 9  | 东西 | 581 |
| 5  | 两个 | 746 | 10 | 告诉 | 570 |

由表 3 可见，前 80 回在有效词数和不同词数上均高于后 40 回，且后 40 回的标准差明显更小，表明其章节长度与词汇丰富度的波动更为集中。由表 4 可见，小说的高频词以“说道”“众人”“姑娘”等叙事性与称谓类词语为主，符合章回体小说的语言特点。

图 1 与图 2 分别展示各章有效词数与词汇丰富度随回目的变化。全书 120 回的有效词数在 890–2 421 之间波动，第 79 回附近出现峰值；不同词数在第 78–79 回达到最高第 20 回附近降至最低。两图均在第 80/81 回分界处标注红色虚线，便于观察前后两部分的整体趋势。



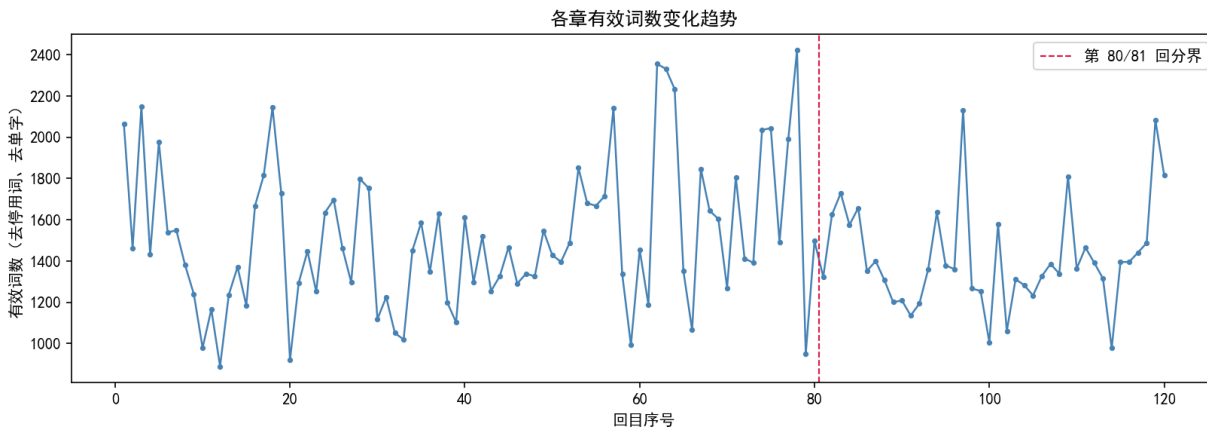


图 1: 各章有效词数变化趋势 (红色虚线为第 80/81 回分界)

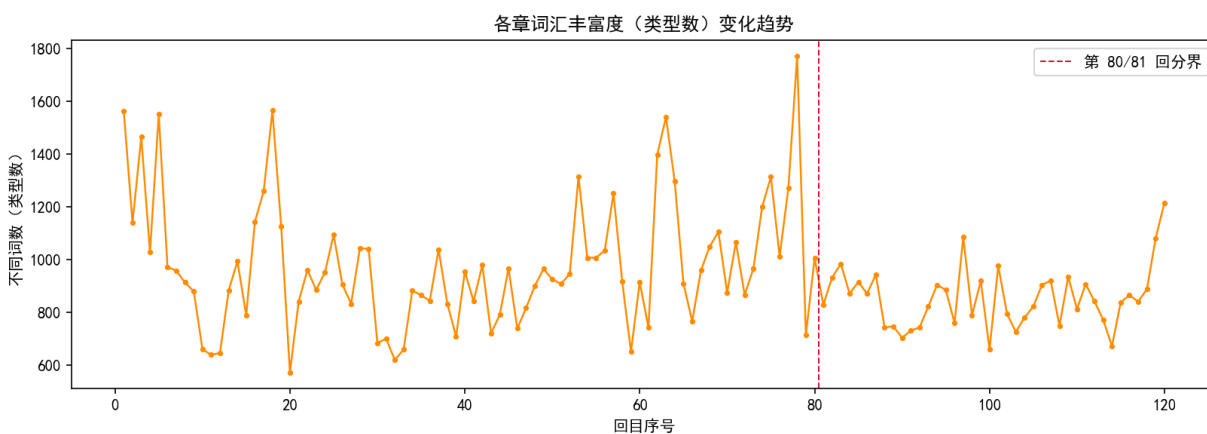


图 2: 各章词汇丰富度 (不同词数) 变化趋势

图 3 展示预处理后 Top 20 高频非人名词及其全书频次, ”说道”以 938 次居首, 其余多为”众人””姑娘””只见”等叙事功能词, 说明特征词以通用表达为主, 而非特定人物名称。

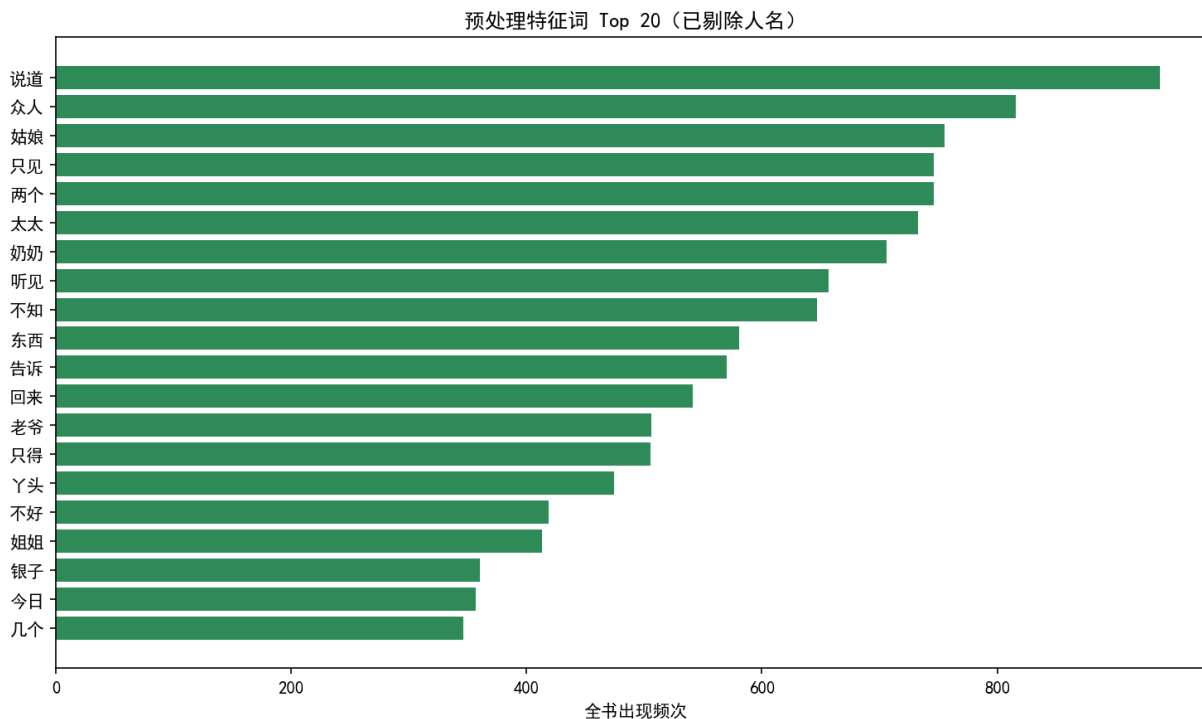


图 3: 预处理特征词 Top 20

图 4 与图 5 从分布与均值两个角度对比前后两部分。箱线图显示：前 80 回在有效词数、不同词数、段落数三项指标上均具有更大的离散程度与若干高值离群点；后 40 回分布更为紧凑，中位数与上四分位数普遍低于前 80 回。均值对比条形图进一步印证：前 80 回的有效词数均值约 1517、不同词数均值约 982、段落数均值约 27，均高于后 40 回对应值。

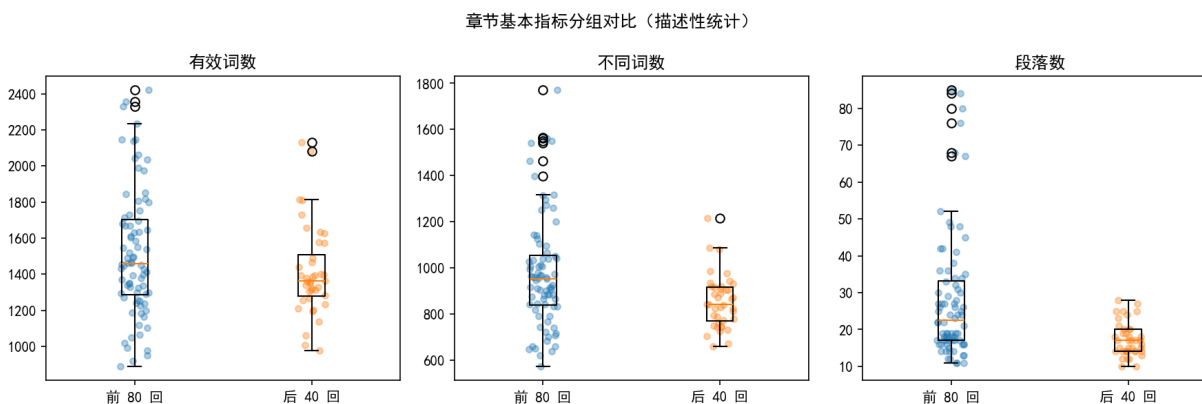


图 4: 前 80 回与后 40 回章节基本指标分组箱线图

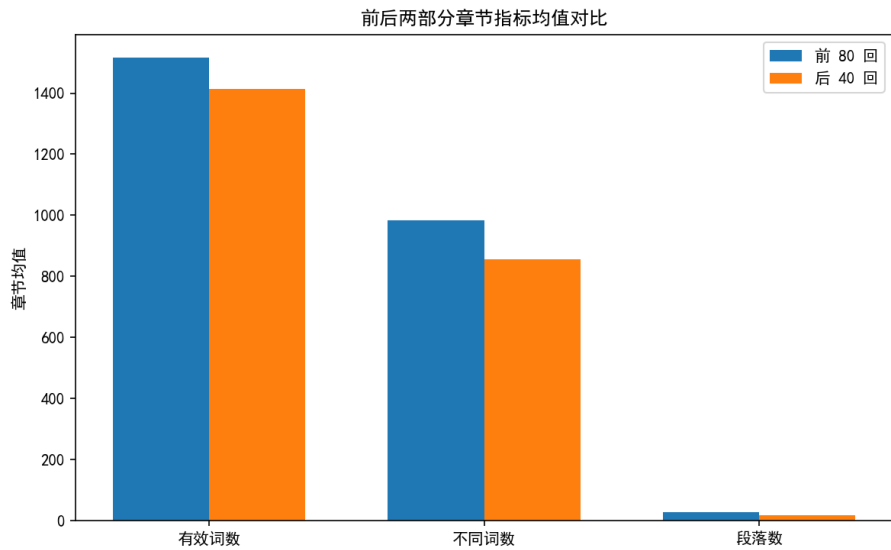


图 5: 前 80 回与后 40 回章节指标均值对比

图 6 为特征矩阵稀疏性热力图，展示 Top 40 高频词在 120 回中的归一化词频分布。颜色越深表示该词在该回中出现频次越高。可见”说道””不知”等词在各回中均有较高出现频率，而部分词语如”老爷””二爷”在特定回目区间更为集中，体现了特征矩阵的稀疏性与词语使用的章节异质性。

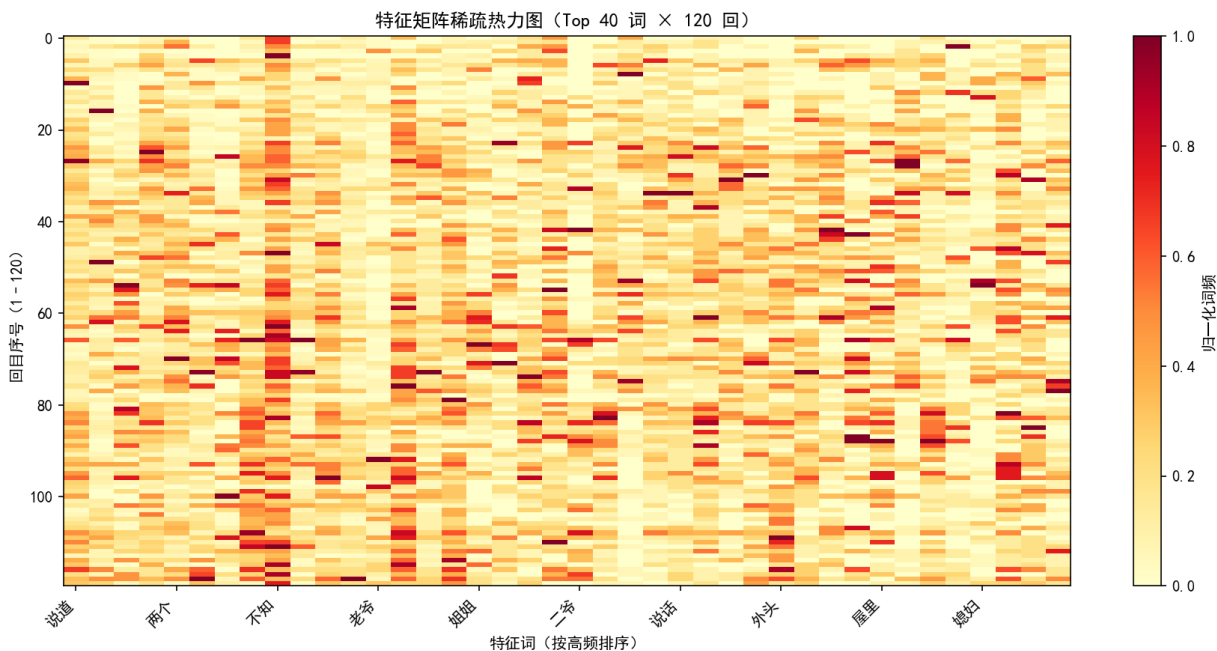


图 6: 特征矩阵稀疏热力图 (Top 40 词 × 120 回，颜色表示归一化词频)

## 4 实验结果与分析

### 4.1 全特征 Mann-Whitney $U$ 检验结果

#### 4.1.1 多重比较校正下的整体显著性

对  $V = 2000$  个特征词逐一进行两独立样本 Mann-Whitney  $U$  检验，并分别施加 Bonferroni 校正与 FDR 校正以控制多重比较带来的假阳性。表 5 汇总了不同校正策略下的显著特征数量。

表 5: 2000 个特征词 Mann-Whitney  $U$  检验显著性汇总 ( $\alpha = 0.05$ )

| 校正方式          | 显著特征数 | 占特征总数比例 |
|---------------|-------|---------|
| 原始检验（未校正）     | 611   | 30.55%  |
| Bonferroni 校正 | 68    | 3.40%   |
| FDR (BH) 校正   | 325   | 16.25%  |

由表 5 可见，即使在最为保守的 Bonferroni 校正下，仍有 68 个词语在前后两部分之间存在显著差异；FDR 校正下显著词语达 325 个，占特征总数的 16.25%。这表明前 80 回与后 40 回之间的用词差异并非个别词语的偶然波动，而是在大量特征维度上具有统计一致性。

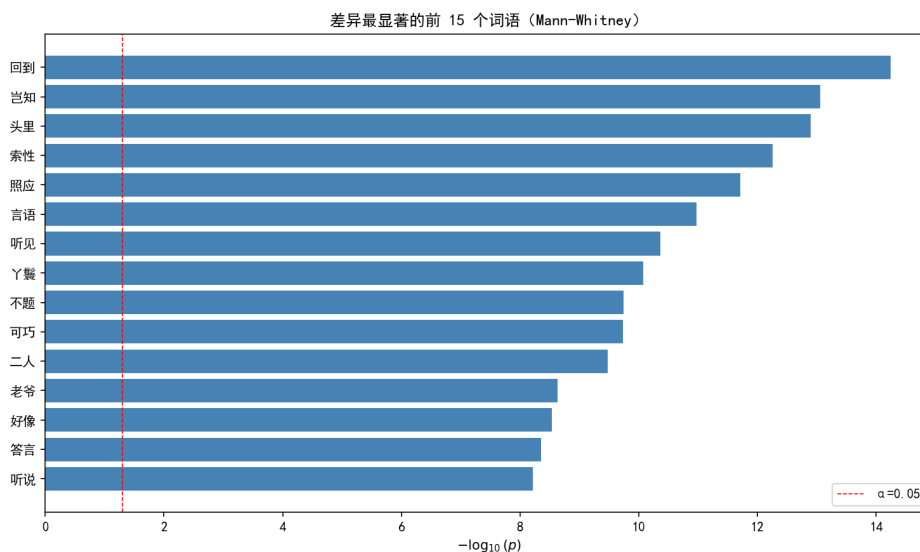


图 7: 差异最显著的前 15 个词语 (Mann-Whitney  $U$  检验,  $-\log_{10}(p)$  条形图)

#### 4.1.2 Top 10 差异词语及效应量

表 6 列出按原始  $p$  值排序的前 10 个差异词语，同时给出 Bonferroni 校正  $p$  值、FDR 校正  $p$  值及 rank-biserial 效应量  $r$ 。 $r > 0$  表示该词在后 40 回中的秩次整体高于前 80 回； $r < 0$  则相反。

表 6: Top 10 差异词语检验结果

| 词语 | $U$ 统计量 | $p$ 值                  | Bonferroni $p$         | FDR $p$                | $r$     |
|----|---------|------------------------|------------------------|------------------------|---------|
| 回到 | 478.0   | $5.77 \times 10^{-15}$ | $1.15 \times 10^{-11}$ | $1.15 \times 10^{-11}$ | 0.7012  |
| 岂知 | 680.0   | $8.81 \times 10^{-14}$ | $1.76 \times 10^{-10}$ | $8.48 \times 10^{-11}$ | 0.5750  |
| 头里 | 472.5   | $1.27 \times 10^{-13}$ | $2.54 \times 10^{-10}$ | $8.48 \times 10^{-11}$ | 0.7047  |
| 索性 | 642.0   | $5.51 \times 10^{-13}$ | $1.10 \times 10^{-9}$  | $2.76 \times 10^{-10}$ | 0.5988  |
| 照应 | 652.0   | $1.95 \times 10^{-12}$ | $3.90 \times 10^{-9}$  | $7.79 \times 10^{-10}$ | 0.5925  |
| 言语 | 447.5   | $1.07 \times 10^{-11}$ | $2.13 \times 10^{-8}$  | $3.55 \times 10^{-9}$  | 0.7203  |
| 听见 | 421.0   | $4.33 \times 10^{-11}$ | $8.66 \times 10^{-8}$  | $1.24 \times 10^{-8}$  | 0.7369  |
| 丫鬟 | 2705.0  | $8.45 \times 10^{-11}$ | $1.69 \times 10^{-7}$  | $2.11 \times 10^{-8}$  | -0.6906 |
| 不题 | 857.5   | $1.78 \times 10^{-10}$ | $3.57 \times 10^{-7}$  | $3.72 \times 10^{-8}$  | 0.4641  |
| 可巧 | 2629.5  | $1.86 \times 10^{-10}$ | $3.72 \times 10^{-7}$  | $3.72 \times 10^{-8}$  | -0.6434 |

从表 6 可归纳两类差异模式：

- **后 40 回显著偏高：**如“回到”“岂知”“头里”“索性”“照应”“言语”“听见”“不题”等， $r$  多在 0.46–0.74 之间，属于中等至较大效应量；
- **前 80 回显著偏高：**如“丫鬟”( $r = -0.6906$ )、“可巧”( $r = -0.6434$ )，在后 40 回中使用频率明显降低。

上述 10 个词语在 Bonferroni 与 FDR 校正下均保持显著，说明差异具有稳健性。

#### 4.1.3 代表性差异词语的分组分布

为更直观展示差异词语在两组间的分布形态，从 FDR 显著词中选取 6 个代表词（“回到”“岂知”“头里”“索性”“照应”“言语”），分别进行 Mann-Whitney  $U$  检验与 Kolmogorov–Smirnov (KS) 两样本检验。结果见表 7。

表 7: 6 个代表性差异词语的补充检验结果

| 词语 | MW $p$ 值               | KS 统计量 $D$ | KS $p$ 值               | $r$    |
|----|------------------------|------------|------------------------|--------|
| 回到 | $5.77 \times 10^{-15}$ | 0.6875     | $1.91 \times 10^{-12}$ | 0.7012 |
| 岂知 | $8.81 \times 10^{-14}$ | 0.5750     | $1.44 \times 10^{-8}$  | 0.5750 |
| 头里 | $1.27 \times 10^{-13}$ | 0.6750     | $5.77 \times 10^{-12}$ | 0.7047 |
| 索性 | $5.51 \times 10^{-13}$ | 0.5875     | $5.94 \times 10^{-9}$  | 0.5988 |
| 照应 | $1.95 \times 10^{-12}$ | 0.5750     | $1.44 \times 10^{-8}$  | 0.5925 |
| 言语 | $1.07 \times 10^{-11}$ | 0.5750     | $1.44 \times 10^{-8}$  | 0.7203 |

6 个代表词的 KS 检验  $p$  值均远小于 0.05, KS 统计量  $D$  在 0.575–0.688 之间, 表明两组在该词归一化词频上的分布形态亦存在显著差异, 而非仅均值或中位数不同。

图 8 以箱线图展示 6 个代表词在前 80 回与后 40 回中的归一化词频分布。可以清晰看到: 前 80 回中”回到””岂知””头里”等词的频次几乎集中于零附近, 后 40 回则出现明显抬升, 且存在高值离群点。

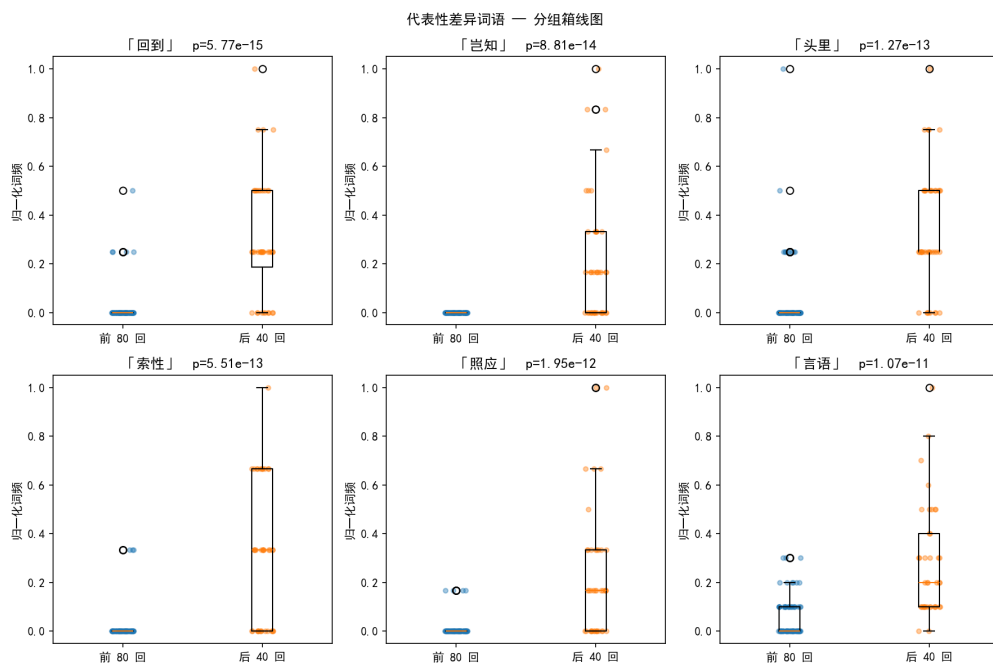


图 8: 6 个代表性差异词语的归一化词频分组箱线图 (前 80 回 vs. 后 40 回)

图 9 进一步以直方图叠加核密度估计曲线对比两组分布。6 个子图的 KS  $p$  值均在  $10^{-8}$ – $10^{-12}$  量级，两组密度曲线几乎无重叠——前 80 回以零频为主，后 40 回呈多峰或宽峰分布，从分布形态上进一步印证了组间差异。

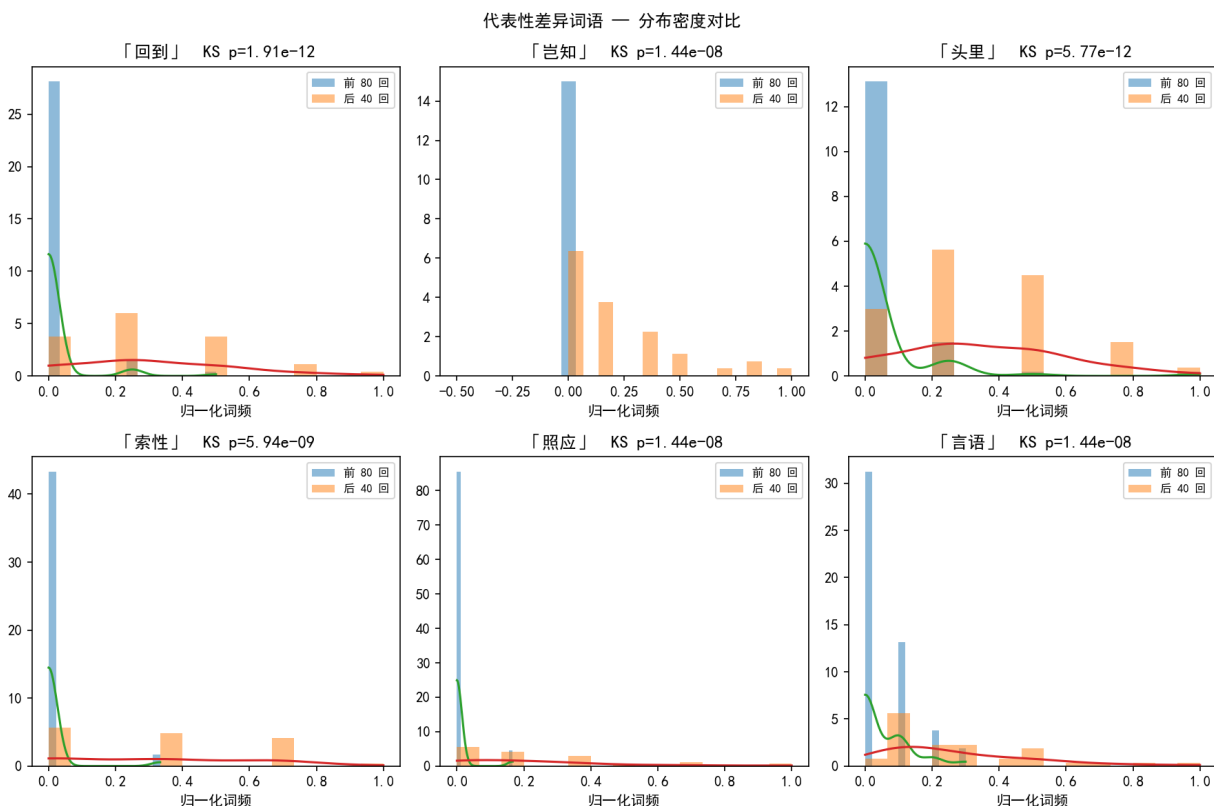


图 9: 6 个代表性差异词语的归一化词频分布密度对比

## 4.2 PCA 降维与 PC1 补充检验

### 4.2.1 主成分方差解释

对  $120 \times 2000$  特征矩阵进行主成分分析，为分析和展示方便，保留前 3 个主成分用于探索性可视化，并展示前 10 个主成分的碎石图。

图 10 的碎石图显示，PC1 以 3.59% 的解释率居首，后续主成分贡献递减平缓，未出现明显“肘部”，说明词频变异在高维空间中较为分散。

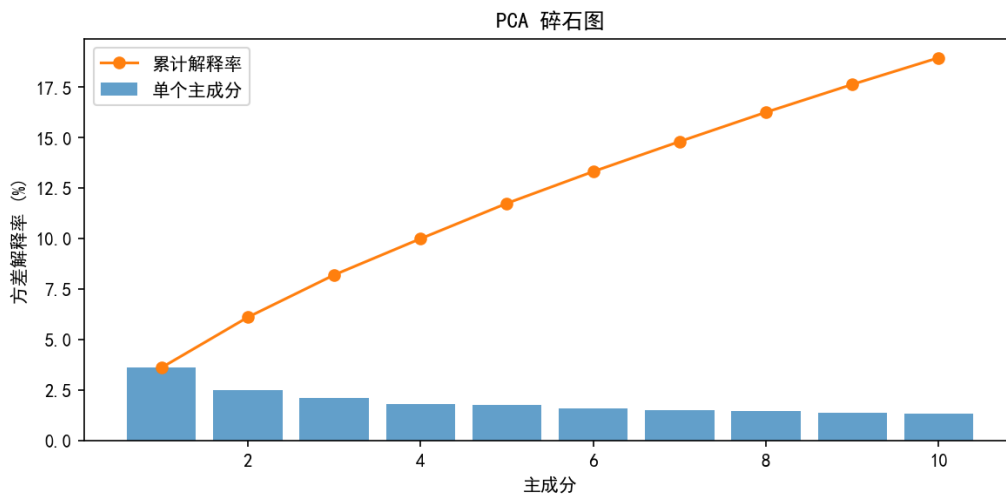


图 10: PCA 碎石图 (前 10 个主成分)

#### 4.2.2 三维散点图的分组聚类趋势

图 11 将 120 回投影至 PC1-PC2-PC3 空间。前 80 回主要聚集在 PC1 负值区域，后 40 回则整体偏移至 PC1 正值区域，两组沿 PC1 轴呈现近乎完全的线性分离，而在 PC2、PC3 方向上存在重叠。

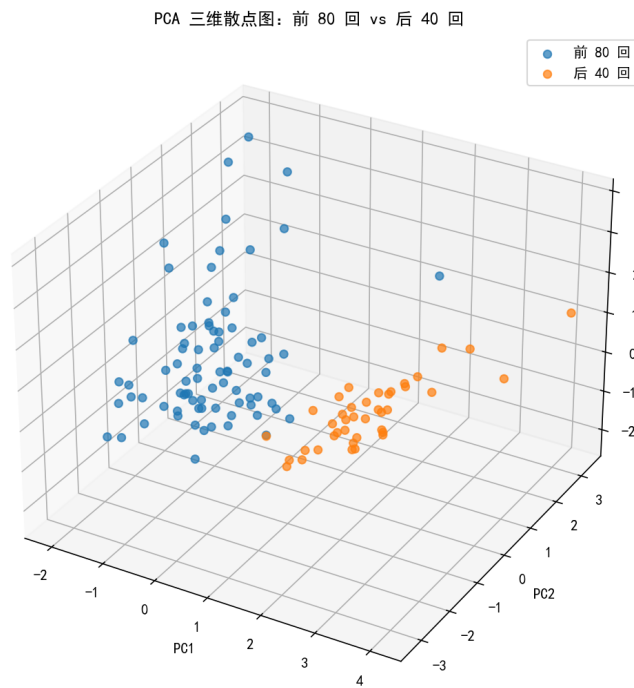


图 11: 前 80 回与后 40 回 PCA 三维散点图



### 4.2.3 PC1 的三项非参数检验

鉴于 PC1 解释方差最大且在三维散点图中表现出最明显的组间分离，对 PC1 得分进一步进行 Mann-Whitney  $U$  检验、Kolmogorov-Smirnov 两样本检验及置换检验（置换 10 000 次）。结果汇总于表 8。

表 8: PC1 得分的三项非参数检验结果

| 检验方法               | 统计量                      | $p$ 值                  | 效应量          |
|--------------------|--------------------------|------------------------|--------------|
| Mann-Whitney $U$   | $U = 12.0$               | $9.78 \times 10^{-19}$ | $r = 0.9925$ |
| Kolmogorov-Smirnov | $D = 0.9750$             | $1.64 \times 10^{-29}$ | —            |
| 置换检验 (10 000 次)    | $T_{\text{obs}} = 3.070$ | $9.999 \times 10^{-5}$ | —            |

三项检验均在  $\alpha = 0.05$  下高度显著。其中 Mann-Whitney 效应量  $r = 0.9925$ ，接近 1，表明 PC1 上两组秩次几乎完全分离；KS 统计量  $D = 0.9750$ ，说明两组 PC1 得分的累积分布函数差异极大。

图 12 的箱线图直观呈现这一分离：前 80 回 PC1 中位数约  $-1.0$ ，IQR 约  $-1.3$  至  $-0.8$ ；后 40 回 PC1 中位数约  $2.0$ ，IQR 约  $1.5$  至  $2.5$ ，两组几乎无重叠区间。

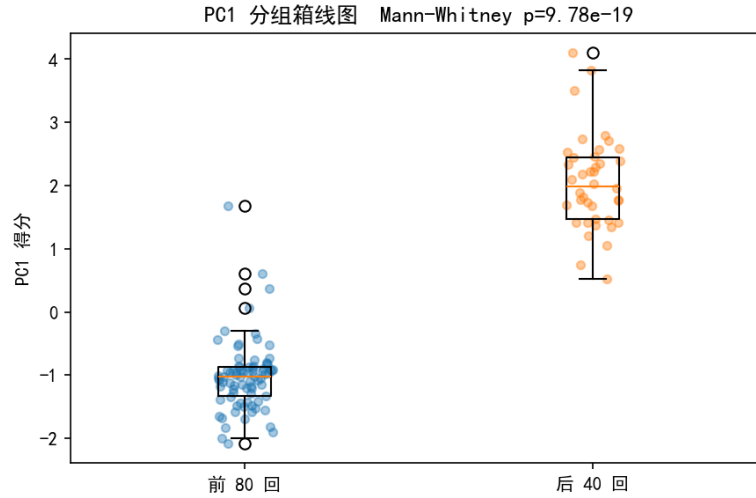


图 12: PC1 得分分组箱线图 (Mann-Whitney  $p = 9.78 \times 10^{-19}$ )

图 13 的密度对比进一步确认：前 80 回 PC1 密度峰位于  $-1.0$  附近，后 40 回峰位于  $+2.0$  附近，KS  $p = 1.64 \times 10^{-29}$ ，两组分布几乎无交叠。

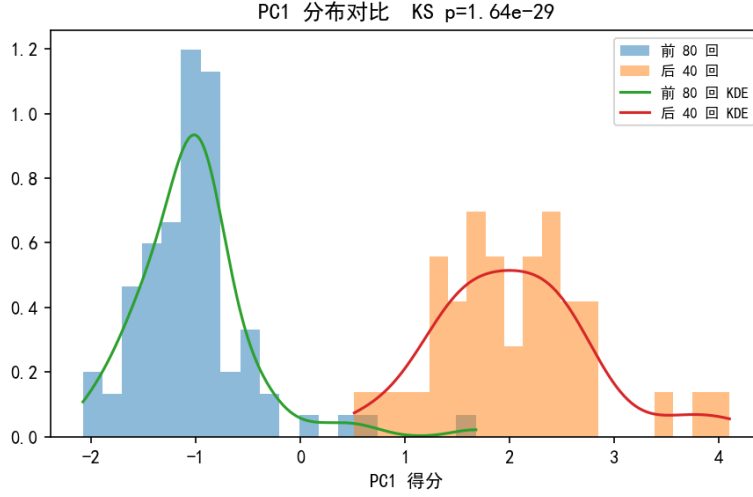


图 13: PC1 得分分布密度对比 (KS  $p = 1.64 \times 10^{-29}$ )

### 4.3 综合分析讨论

综合逐词 Mann-Whitney 检验与 PCA/PC1 补充检验的结果，在  $\alpha = 0.05$  下可得到以下结论：

1. **差异具有广泛性。**2000 个特征词中，611 个在原始检验下显著；即使经 Bonferroni 严格校正，仍有 68 个词语显著，FDR 校正下达 325 个。这说明前 80 回与后 40 回之间的用词差异并非孤立现象，而是在相当比例的特征维度上同时存在。
2. **差异具有方向性。**差异词语可明确分为”后 40 回偏高”（如”回到””岂知””头里””索性”等）与”前 80 回偏高”（如”丫鬟””可巧”）两类，反映的不仅是某一方向的偏移，而是整体用词习惯的多维变化。
3. **整体风格在低维投影中可分离。**PCA 三维散点图显示两组沿 PC1 轴近乎完全分离；PC1 上 Mann-Whitney ( $p = 9.78 \times 10^{-19}$ ,  $r = 0.9925$ )、KS ( $p = 1.64 \times 10^{-29}$ ) 及置换检验 ( $p < 10^{-4}$ ) 均高度显著，从全局维度验证了组间风格差异。

需要指出的是，PCA 前 3 个主成分累计仅解释 8.19% 的总方差，说明 2000 维词频空间的信息高度分散，单一或少数主成分无法完整刻画全部风格变异。因此，本研究以**全特征逐词 Mann-Whitney 检验**作为核心推断手段，以**PCA/PC1 检验**作为整体趋势的可视化与补充验证，两者结论相互印证。

# A 实验代码

## A.1 数据获取

```
1 import requests
2 import re
3 import os
4 from bs4 import BeautifulSoup
5 from lxml import etree # noqa: F401 — 供 BeautifulSoup(..., 'lxml') 解析器使用
6 from tqdm import tqdm
7
8 # =====
9 # 1. 初始化: 若不存在则创建本地存储目录
10 # =====
11 if not os.path.exists('红楼梦'):
12     os.mkdir('红楼梦')
13
14 # =====
15 # 2. 抓取索引页: 解析 120 回的目录标题与正文链接
16 # =====
17 url = 'http://www.gudianmingzhu.com/guji/hongloumeng/'
18
19 # 设置 User-Agent 伪装浏览器, 避免被目标站点拒绝访问
20 headers = {
21     'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/87.0.4280.141 Safari/537.36'
22 }
23
24 page_text = requests.get(url=url, headers=headers)
25 page_text.encoding = page_text.apparent_encoding # 自动识别响应编码
26 page_text = page_text.text
27
28 soup = BeautifulSoup(page_text, 'lxml')
29 aTagList = soup.select('#leftdg > div.dgcon.ui > div > a') # 目录区全部章节 <a> 标签
30 titleList = [i.text for i in aTagList] # 各回在目录中显示的标题
31 urlList = [i["href"] for i in aTagList] # 各回正文页面的 URL
32
33 # =====
34 # 3. 逐回抓取正文、清洗文本并写入本地
35 # =====
36 progress_bar = tqdm(total=120, desc='抓取章节')
37
38 for index in range(1, 121):
39     progress_bar.update(1)
40
41     title = titleList[index - 1]
42     url = urlList[index - 1]
43
44     # 请求单章页面 (超时 10 秒)
45     page_text = requests.get(url=url, headers=headers, timeout=10)
46     page_text.encoding = page_text.apparent_encoding
47     page_text = page_text.text
48
49     soup = BeautifulSoup(page_text, 'lxml')
50
51     # 定位正文容器, 拼接所有段落 <p> 的纯文本
52     tmp = soup.select('#leftdg > div:nth-child(1) > div > div')
53     content = ''.join([p.text for result in tmp for p in result.find_all('p')])
54
55     # 正文首段通常包含完整回目标题
56     titleText = soup.select('#leftdg > div:nth-child(1) > div > div > p:nth-child(1)')[0].text
57
58     # 将连续两个全角空格 (U+3000) 替换为换行符, 保留段落换行
59     pattern = re.compile(r'\u3000{2}')
60     content = re.sub(pattern, '\n', content)
61
62     # 去除标题中的全角空格, 避免文件名含不可见字符
63     titleText = titleText.replace('\u3000', '')
64
65     # 合并目录标题与页面内标题, 构成完整文件名
```

```

66     title = titleList[index - 1] + titleText
67
68     chapter_path = '红楼梦/{0}_{0}.txt'.format(index, title)
69
70     # 若同名文件已存在则先删除，再写入最新抓取内容
71     if os.path.exists(chapter_path):
72         os.remove(chapter_path)
73
74     with open(chapter_path, 'w', encoding='utf-8') as f:
75         f.write(content)
76
77 progress_bar.close()

```

## A.2 数据预处理与可视化

```

1  import itertools
2  import json
3  import os
4  import re
5
6  import jieba
7  import matplotlib.pyplot as plt
8  import numpy as np
9  from mpl_toolkits.mplot3d import Axes3D # noqa: F401
10 from scipy import stats
11 from sklearn import decomposition
12 from tqdm import tqdm
13
14 # =====
15 # 全局配置
16 # =====
17 V = 2000 # 每章特征维度：取高频非人名词 Top V
18 N_FRONT = 80 # 前 80 回为对照组，后 40 回为比较组
19 N_TOP_WORDS_PLOT = 20 # 描述性统计：高频词条形图展示数量
20 N_REP_FEATURES = 6 # 代表词数量（用于箱线图/密度图）
21 N_COMPONENTS = 3 # PCA 保留主成分数
22 N_PERM = 10000 # PC1 置换检验重复次数
23 SCREE_COMPONENTS = 10 # 碎石图展示的主成分个数
24 ALPHA = 0.05 # 显著性水平
25
26 OUTPUT_DIR = 'output'
27 REPORT_PATH = os.path.join(OUTPUT_DIR, 'report.txt')
28 CHAPTER_DIR = '红楼梦'
29 USERDICT_PATH = 'userdict.json'
30 STOPWORDS_PATH = 'stopwords.txt'
31 SHOW_PLOTS = False
32
33 plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'DejaVu Sans']
34 plt.rcParams['axes.unicode_minus'] = False
35
36
37 # =====
38 # 第一部分：文本预处理与特征构建
39 # =====
40 def load_names(file_path=USERDICT_PATH):
41     """加载人物别名表，并注册到 jieba 词典（标记为人名 nr）。"""
42     with open(file_path, 'r', encoding='utf-8') as f:
43         names = json.load(f)
44     for name in names:
45         for alias_name in names[name]:
46             jieba.add_word(alias_name, tag='nr')
47     return names
48
49
50 def load_stopwords(file_path=STOPWORDS_PATH):
51     """加载停用词表。"""
52     with open(file_path, 'r', encoding='utf-8') as f:
53         return [line.strip('\n') for line in f.readlines()]
54
55

```

```

56 def tokenize_chapter(file_path, stopwords):
57     """对单章文本分词，过滤单字词与停用词，返回段落级词列表。"""
58     with open(file_path, 'rb') as f:
59         content = f.read()
60     para = content.decode('utf-8', 'ignore').split('\n')[:-1]
61     result = []
62     for p in para:
63         words = []
64         for x in jieba.cut(p, cut_all=False):
65             if len(x) <= 1 or x in stopwords:
66                 continue
67             words.append(x)
68         result.append(words)
69     return result
70
71
72 def tokenize_all_chapters(chapter_dir=CHAPTER_DIR):
73     """遍历各章分词，返回 {章节号: [[段1词列表], ...]}。"""
74     stopwords = load_stopwords()
75     result = {}
76     for file_name in tqdm(os.listdir(chapter_dir), desc='分词'):
77         index = int(file_name.split('_')[0])
78         result[index] = tokenize_chapter(os.path.join(chapter_dir, file_name), stopwords)
79     return result
80
81
82 def build_word_freq(tokenized):
83     """统计全书词频与各章词频。"""
84     artical_word_dict = {}
85     chapters_word_dict = {}
86     for index in tokenized:
87         chapter_word_list = list(itertools.chain.from_iterable(tokenized[index]))
88         chapter_word_dict = {}
89         for word in chapter_word_list:
90             chapter_word_dict[word] = chapter_word_dict.get(word, 0) + 1
91             artical_word_dict[word] = artical_word_dict.get(word, 0) + 1
92         chapters_word_dict[index] = chapter_word_dict
93     sorted_word_list = [
94         word for _, word in sorted(
95             zip(artical_word_dict.values(), artical_word_dict.keys()), reverse=True)
96     ]
97     return chapters_word_dict, sorted_word_list, artical_word_dict
98
99
100 def remove_person_names(sorted_word_list, names):
101     """从高频词列表中剔除人物名，避免人名主导风格特征。"""
102     all_names = [alias for aliases in names.values() for alias in aliases]
103     filtered = []
104     for word in sorted_word_list:
105         is_name = any(re.search(re.compile(f'(.*){name}(.*)'), word) for name in all_names)
106         if not is_name:
107             filtered.append(word)
108     return filtered
109
110
111 def build_feature_matrix(chapters_word_dict, word_list):
112     """
113     构建章节 × 词语 特征矩阵 (120 × V)。
114     行按章节号 1-120 排序；列按 word_list 顺序；值为归一化词频。
115     """
116     chapter_ids = sorted(chapters_word_dict.keys())
117     features = np.zeros((len(chapter_ids), len(word_list)))
118     for i, chap_id in enumerate(chapter_ids):
119         chapter_word_dict = chapters_word_dict[chap_id]
120         for j, word in enumerate(word_list):
121             features[i, j] = chapter_word_dict.get(word, 0)
122     # 按列归一化到 [0, 1]，消除词频绝对量级差异
123     col_max = features.max(axis=0)
124     col_max[col_max == 0] = 1
125     features /= col_max
126     return features, chapter_ids
127
128
129 def compute_chapter_stats(tokenized):

```

```

130     """汇总各章分词后的基本计数指标。"""
131     chapter_stats = []
132     for chap_id in sorted(tokenized.keys()):
133         words = list(itertools.chain.from_iterable(tokenized[chap_id]))
134         chapter_stats.append({
135             'chapter_id': chap_id,
136             'token_count': len(words),
137             'unique_count': len(set(words)),
138             'paragraph_count': len(tokenized[chap_id]),
139         })
140     return chapter_stats
141
142
143 def prepare_features():
144     """预处理流水线：分词 → 词频 → 去人名 → 特征矩阵。"""
145     names = load_names()
146     tokenized = tokenize_all_chapters()
147     chapters_word_dict, sorted_word_list, artical_word_dict = build_word_freq(tokenized)
148     filtered_words = remove_person_names(sorted_word_list, names)
149     word_list = filtered_words[:V]
150     features, chapter_ids = build_feature_matrix(chapters_word_dict, word_list)
151     chapter_stats = compute_chapter_stats(tokenized)
152     preprocess_meta = {
153         'chapter_stats': chapter_stats,
154         'book_word_freq': artical_word_dict,
155         'sorted_word_list': sorted_word_list,
156         'filtered_words': filtered_words,
157         'vocab_before_filter': len(sorted_word_list),
158         'vocab_after_filter': len(filtered_words),
159         'names_count': sum(len(aliaes) for aliaes in names.values()),
160     }
161     return features, word_list, chapter_ids, preprocess_meta
162
163 # =====
164 # 第二部分：描述性统计与预处理可视化
165 # =====
166 def _save_fig(name):
167     """保存当前图像到 output/ 目录。"""
168     path = os.path.join(OUTPUT_DIR, name)
169     plt.tight_layout()
170     plt.savefig(path, dpi=150, bbox_inches='tight')
171     plt.close()
172     return path
173
174 def _group_split(values, n_front=N_FRONT):
175     """将按章节排序的一维数组拆分为前 80 回与后 40 回。"""
176     return values[:n_front], values[n_front:]
177
178
179 def _describe_array(arr):
180     """计算一组样本的常见描述统计量。"""
181     arr = np.asarray(arr, dtype=float)
182     return {
183         'n': len(arr),
184         'mean': float(np.mean(arr)),
185         'std': float(np.std(arr, ddof=1)) if len(arr) > 1 else 0.0,
186         'median': float(np.median(arr)),
187         'min': float(np.min(arr)),
188         'max': float(np.max(arr)),
189         'q1': float(np.percentile(arr, 25)),
190         'q3': float(np.percentile(arr, 75)),
191     }
192
193
194 def compute_descriptive_stats(preprocess_meta, features, word_list, chapter_ids, n_front=N_FRONT):
195     """汇总预处理后的描述性统计结果。"""
196     chapter_stats = preprocess_meta['chapter_stats']
197     token_counts = np.array([s['token_count'] for s in chapter_stats], dtype=float)
198     unique_counts = np.array([s['unique_count'] for s in chapter_stats], dtype=float)
199     para_counts = np.array([s['paragraph_count'] for s in chapter_stats], dtype=float)
200
201     front_tokens, back_tokens = _group_split(token_counts, n_front)
202     front_unique, back_unique = _group_split(unique_counts, n_front)
203     front_para, back_para = _group_split(para_counts, n_front)

```

```

204 nonzero_ratio = float(np.mean(features > 0))
205
206
207 top_words = []
208 for word in preprocess_meta['filtered_words'][:N_TOP_WORDS_PLOT]:
209     top_words.append({
210         'word': word,
211         'freq': preprocess_meta['book_word_freq'].get(word, 0),
212     })
213
214 return {
215     'n_chapters': len(chapter_ids),
216     'n_features': len(word_list),
217     'total_tokens': int(token_counts.sum()),
218     'book_vocab': len(preprocess_meta['book_word_freq']),
219     'vocab_before_filter': preprocess_meta['vocab_before_filter'],
220     'vocab_after_filter': preprocess_meta['vocab_after_filter'],
221     'names_registered': preprocess_meta['names_count'],
222     'feature_nonzero_ratio': nonzero_ratio,
223     'chapter_token_stats': _describe_array(token_counts),
224     'chapter_unique_stats': _describe_array(unique_counts),
225     'chapter_para_stats': _describe_array(para_counts),
226     'front_token_stats': _describe_array(front_tokens),
227     'back_token_stats': _describe_array(back_tokens),
228     'front_unique_stats': _describe_array(front_unique),
229     'back_unique_stats': _describe_array(back_unique),
230     'front_para_stats': _describe_array(front_para),
231     'back_para_stats': _describe_array(back_para),
232     'top_words': top_words,
233     'chapter_ids': chapter_ids,
234     'token_counts': token_counts,
235     'unique_counts': unique_counts,
236     'paragraph_counts': para_counts,
237 }
238
239
240 def plot_chapter_token_trend(desc_stats, n_front=N_FRONT):
241     """各章有效词数随回目变化折线图。"""
242     chapter_ids = desc_stats['chapter_ids']
243     token_counts = desc_stats['token_counts']
244
245     fig, ax = plt.subplots(figsize=(11, 4))
246     ax.plot(chapter_ids, token_counts, color='steelblue', linewidth=1.2, marker='o', markersize=2.5)
247     ax.axvline(n_front + 0.5, color='crimson', linestyle='--', linewidth=1.0, label='第 80/81 回分界')
248     ax.set_xlabel('回目序号')
249     ax.set_ylabel('有效词数 (去停用词、去单字)')
250     ax.set_title('各章有效词数变化趋势')
251     ax.legend()
252     return _save_fig('desc_chapter_token_trend.png')
253
254
255 def plot_chapter_unique_trend(desc_stats, n_front=N_FRONT):
256     """各章不同词数 (类型数) 随回目变化折线图。"""
257     chapter_ids = desc_stats['chapter_ids']
258     unique_counts = desc_stats['unique_counts']
259
260     fig, ax = plt.subplots(figsize=(11, 4))
261     ax.plot(chapter_ids, unique_counts, color='darkorange', linewidth=1.2, marker='o', markersize=2.5)
262     ax.axvline(n_front + 0.5, color='crimson', linestyle='--', linewidth=1.0, label='第 80/81 回分界')
263     ax.set_xlabel('回目序号')
264     ax.set_ylabel('不同词数 (类型数)')
265     ax.set_title('各章词汇丰富度 (类型数) 变化趋势')
266     ax.legend()
267     return _save_fig('desc_chapter_unique_trend.png')
268
269
270 def plot_top_words_bar(desc_stats):
271     """全书 Top 高频非人名词条条形图。"""
272     words = [item['word'] for item in desc_stats['top_words']]
273     freqs = [item['freq'] for item in desc_stats['top_words']]
274
275     fig, ax = plt.subplots(figsize=(10, 6))
276     y_pos = np.arange(len(words))
277     ax.barh(y_pos, freqs, color='seagreen')

```

```

278 ax.set_yticks(y_pos)
279 ax.set_yticklabels(words)
280 ax.invert_yaxis()
281 ax.set_xlabel('全书出现频次')
282 ax.set_title(f'预处理特征词 Top {len(words)} (已删除人名)')
283 return _save_fig('desc_top_words.png')
284
285
286 def plot_chapter_stats_boxplot(desc_stats, n_front=N_FRONT):
287     """前 80 回 vs 后 40 回：词数、类型数、段落数箱线图。"""
288     metrics = [
289         ('token_counts', '有效词数'),
290         ('unique_counts', '不同词数'),
291         ('paragraph_counts', '段落数'),
292     ]
293     fig, axes = plt.subplots(1, 3, figsize=(12, 4))
294     for ax, (key, label) in zip(axes, metrics):
295         values = desc_stats[key]
296         front, back = _group_split(values, n_front)
297         ax.boxplot([front, back], tick_labels=['前 80 回', '后 40 回'])
298         jitter_front = np.random.default_rng(0).uniform(-0.08, 0.08, len(front))
299         jitter_back = np.random.default_rng(1).uniform(-0.08, 0.08, len(back))
300         ax.scatter(1 + jitter_front, front, alpha=0.35, s=18, color='C0')
301         ax.scatter(2 + jitter_back, back, alpha=0.35, s=18, color='C1')
302         ax.set_title(label)
303     fig.suptitle('章节基本指标分组对比 (描述性统计)')
304     return _save_fig('desc_chapter_stats_boxplot.png')
305
306
307 def plot_group_mean_comparison(desc_stats):
308     """前 80 回 vs 后 40 回：关键指标均值对比条形图。"""
309     labels = ['有效词数', '不同词数', '段落数']
310     front_means = [
311         desc_stats['front_token_stats']['mean'],
312         desc_stats['front_unique_stats']['mean'],
313         desc_stats['front_para_stats']['mean'],
314     ]
315     back_means = [
316         desc_stats['back_token_stats']['mean'],
317         desc_stats['back_unique_stats']['mean'],
318         desc_stats['back_para_stats']['mean'],
319     ]
320
321     x = np.arange(len(labels))
322     width = 0.35
323     fig, ax = plt.subplots(figsize=(8, 5))
324     ax.bar(x - width / 2, front_means, width, label='前 80 回', color='C0')
325     ax.bar(x + width / 2, back_means, width, label='后 40 回', color='C1')
326     ax.set_xticks(x)
327     ax.set_xticklabels(labels)
328     ax.set_ylabel('章节均值')
329     ax.set_title('前后两部分章节指标均值对比')
330     ax.legend()
331     return _save_fig('desc_group_mean_comparison.png')
332
333
334 def plot_feature_sparsity_heatmap(features, word_list, max_words=40):
335     """特征矩阵稀疏模式热力图 (展示前若干高频词在各章的使用情况)。"""
336     n_words = min(max_words, features.shape[1])
337     data = features[:, :n_words]
338
339     fig, ax = plt.subplots(figsize=(12, 6))
340     im = ax.imshow(data, aspect='auto', cmap='YlOrRd', interpolation='nearest')
341     ax.set_xlabel('特征词 (按高频排序)')
342     ax.set_ylabel('回目序号 (1-120)')
343     ax.set_title(f'特征矩阵稀疏热力图 (Top {n_words} 词 × 120 回)')
344     tick_step = max(1, n_words // 10)
345     ax.set_xticks(np.arange(0, n_words, tick_step))
346     ax.set_xticklabels([word_list[i] for i in range(0, n_words, tick_step)], rotation=45, ha='right')
347     fig.colorbar(im, ax=ax, label='归一化词频')
348     return _save_fig('desc_feature_sparsity_heatmap.png')
349
350
351 def run_preprocessing_visualization(desc_stats, features, word_list):

```



```

352     """生成全部预处理描述性统计图表。"""
353     return [
354         plot_chapter_token_trend(desc_stats),
355         plot_chapter_unique_trend(desc_stats),
356         plot_top_words_bar(desc_stats),
357         plot_chapter_stats_boxplot(desc_stats),
358         plot_group_mean_comparison(desc_stats),
359         plot_feature_sparsity_heatmap(features, word_list),
360     ]
361
362
363 def format_descriptive_report(desc_stats):
364     """格式化描述性统计文本报告。"""
365     sep = '=' * 60
366     lines = []
367
368     def add(text=''):
369         lines.append(text)
370
371     def fmt_stats(title, stats_dict):
372         add(f' {title}:')
373         add(f'      n={stats_dict["n"]}, 均值={stats_dict["mean"]:.2f}, '
374            f'标准差={stats_dict["std"]:.2f}, 中位数={stats_dict["median"]:.2f}')
375         add(f'      最小值={stats_dict["min"]:.0f}, Q1={stats_dict["q1"]:.2f}, '
376            f'Q3={stats_dict["q3"]:.2f}, 最大值={stats_dict["max"]:.0f}')
377
378     add(sep)
379     add('描述性统计 (预处理后) ')
380     add(sep)
381     add(f' 章节总数: {desc_stats["n_chapters"]}')
382     add(f' 全书有效词次总数: {desc_stats["total_tokens"]}')
383     add(f' 全书不同词数 (词汇量): {desc_stats["book_vocab"]}')
384     add(f' 去人名前不同词数: {desc_stats["vocab_before_filter"]}')
385     add(f' 去人名后不同词数: {desc_stats["vocab_after_filter"]}')
386     add(f' 特征维度 V: {desc_stats["n_features"]}')
387     add(f' 特征矩阵非零占比: {desc_stats["feature_nonzero_ratio"]:.4f}')
388     add(f' 注册人物别名数: {desc_stats["names_registered"]}')
389
390     add('\n [各章有效词数 — 全书]')
391     fmt_stats('有效词数', desc_stats['chapter_token_stats'])
392     add('\n [各章不同词数 — 全书]')
393     fmt_stats('不同词数', desc_stats['chapter_unique_stats'])
394     add('\n [各章段落数 — 全书]')
395     fmt_stats('段落数', desc_stats['chapter_para_stats'])
396
397     add('\n [前 80 回 vs 后 40 回 — 有效词数]')
398     fmt_stats('前 80 回', desc_stats['front_token_stats'])
399     fmt_stats('后 40 回', desc_stats['back_token_stats'])
400     add('\n [前 80 回 vs 后 40 回 — 不同词数]')
401     fmt_stats('前 80 回', desc_stats['front_unique_stats'])
402     fmt_stats('后 40 回', desc_stats['back_unique_stats'])
403
404     add('\n Top 10 高频非人名词:')
405     add(f' {"词语":<12} {"全书频次":>10}')
406     for item in desc_stats['top_words'][:10]:
407         add(f' {item["word"]:<12} {item["freq"]:>10}')
408
409     return '\n'.join(lines)

```

## A.3 全特征非参数检验

```

1  # =====
2  # 第三部分: 全特征非参数检验 (核心推断)
3  # =====
4  def rank_biserial(u_stat, n1, n2):
5      """Mann-Whitney U 检验的 rank-biserial 效应量。"""
6      return 1 - (2 * u_stat) / (n1 * n2)
7
8
9  def mann_whitney_two_sample(x, y):

```

```

10     """两独立样本 Mann-Whitney U 检验。"""
11     res = stats.mannwhitneyu(x, y, alternative='two-sided')
12     r = rank_biserial(res.statistic, len(x), len(y))
13     return {'U': res.statistic, 'p': res.pvalue, 'rank_biserial_r': r}
14
15
16 def ks_two_sample(x, y):
17     """两独立样本 Kolmogorov-Smirnov 检验。"""
18     res = stats.ks_2samp(x, y)
19     return {'statistic': res.statistic, 'p': res.pvalue}
20
21
22 def feature_wise_tests(features, word_list, n_front=N_FRONT, alpha=ALPHA):
23     """
24     对全部 V 个词频特征逐一做 Mann-Whitney U 检验，
25     并进行 Bonferroni 与 FDR (BH) 多重比较校正。
26     另对 Top 代表词补充 KS 检验，供可视化解读。
27     """
28     x_front = features[:n_front]
29     x_back = features[n_front:]
30     n_features = features.shape[1]
31     u_stats = np.zeros(n_features)
32     p_values = np.zeros(n_features)
33     effect_sizes = np.zeros(n_features)
34
35     for j in tqdm(range(n_features), desc='逐词 Mann-Whitney'):
36         res = stats.mannwhitneyu(x_front[:, j], x_back[:, j], alternative='two-sided')
37         u_stats[j] = res.statistic
38         p_values[j] = res.pvalue
39         effect_sizes[j] = rank_biserial(res.statistic, n_front, len(x_back))
40
41     bonferroni_p = np.minimum(p_values * n_features, 1.0)
42     fdr_p = stats.false_discovery_control(p_values, method='bh')
43
44     sig_raw = int(np.sum(p_values < alpha))
45     sig_bonf = int(np.sum(bonferroni_p < alpha))
46     sig_fdr = int(np.sum(fdr_p < alpha))
47
48     # Top 差异词 (按原始 p 值)
49     order = np.argsort(p_values)
50     top_records = []
51     for idx in order[:15]:
52         top_records.append({
53             'word': word_list[idx],
54             'U': u_stats[idx],
55             'p': p_values[idx],
56             'bonferroni_p': bonferroni_p[idx],
57             'fdr_p': fdr_p[idx],
58             'rank_biserial_r': effect_sizes[idx],
59         })
60
61     # FDR 显著词
62     fdr_sig_indices = np.where(fdr_p < alpha)[0]
63     fdr_sig_sorted = fdr_sig_indices[np.argsort(p_values[fdr_sig_indices])]
64     top_fdr_words = [
65         {
66             'word': word_list[idx],
67             'p': p_values[idx],
68             'fdr_p': fdr_p[idx],
69             'rank_biserial_r': effect_sizes[idx],
70         }
71         for idx in fdr_sig_sorted[:15]
72     ]
73
74     # 代表词: 优先从 FDR 显著词中选取, 不足则回退到原始 p 值 Top 词
75     rep_source = top_fdr_words if top_fdr_words else top_records
76     representative = []
77     seen_words = set()
78     for rec in rep_source:
79         word = rec['word']
80         if word in seen_words:
81             continue
82         seen_words.add(word)
83         idx = word_list.index(word)

```

```

84     front_vals = x_front[:, idx]
85     back_vals = x_back[:, idx]
86     mw = mann_whitney_two_sample(front_vals, back_vals)
87     ks = ks_two_sample(front_vals, back_vals)
88     representative.append({
89         'word': word,
90         'index': idx,
91         'U': mw['U'],
92         'p': rec.get('p', p_values[idx]),
93         'bonferroni_p': bonferroni_p[idx],
94         'fdr_p': rec.get('fdr_p', fdr_p[idx]),
95         'rank_biserial_r': mw['rank_biserial_r'],
96         'ks_D': ks['statistic'],
97         'ks_p': ks['p'],
98     })
99     if len(representative) >= N_REP_FEATURES:
100         break
101
102     return {
103         'p_values': p_values,
104         'bonferroni_p': bonferroni_p,
105         'fdr_p': fdr_p,
106         'effect_sizes': effect_sizes,
107         'sig_raw': sig_raw,
108         'sig_bonferroni': sig_bonf,
109         'sig_fdr': sig_fdr,
110         'top_records': top_records,
111         'top_fdr_words': top_fdr_words,
112         'representative': representative,
113     }

```

## A.4 主成分分析、补充检验与剩余代码

```

1  # =====
2  # 第四部分：PCA 降维 + PC1 补充检验
3  # =====
4
5  def run_pca(features, n_components=N_COMPONENTS, scree_components=SCREE_COMPONENTS):
6      """
7      对特征矩阵做 PCA 降维。
8      返回：前 n_components 维得分、所选成分解释率、完整碎石图数据。
9      """
10     n_scree = min(scree_components, features.shape[0], features.shape[1])
11     pca_full = decomposition.PCA(n_components=n_scree)
12     pca_full.fit(features)
13     scores = pca_full.transform(features)[: , :n_components]
14     return scores, pca_full.explained_variance_ratio_[:n_components], pca_full.explained_variance_ratio_
15
16
17  def _pc1_mean_diff(pc1, labels):
18      """PC1 上两组样本均值的绝对差，用作置换检验统计量。"""
19     front_mean = pc1[labels == 0].mean()
20     back_mean = pc1[labels == 1].mean()
21     return abs(front_mean - back_mean)
22
23
24  def permutation_test_pc1(scores, n_front=N_FRONT, n_perm=N_PERM, seed=42):
25      """
26      PC1 置换检验：在原标签下计算两组 PC1 均值差，
27      重复打乱组标签，构造零分布并计算 p 值。
28      """
29     rng = np.random.default_rng(seed)
30     pc1 = scores[:, 0]
31     labels = np.array([0] * n_front + [1] * (len(scores) - n_front))
32     t_obs = _pc1_mean_diff(pc1, labels)
33     count = 0
34     for _ in range(n_perm):
35         perm_labels = rng.permutation(labels)
36         if _pc1_mean_diff(pc1, perm_labels) >= t_obs:
37             count += 1

```

```

38     p_value = (count + 1) / (n_perm + 1)
39     return {'T_obs': t_obs, 'p': p_value, 'n_perm': n_perm}
40
41
42 def pci_tests(scores, n_front=N_FRONT):
43     """
44     对第一主成分 (PC1) 做三项非参数检验:
45     Mann-Whitney U, Kolmogorov-Smirnov, 置换检验。
46     PC1 解释方差最大, 作为 PCA 方向的代表性检验。
47     """
48     pci_front = scores[:n_front, 0]
49     pci_back = scores[n_front:, 0]
50     return {
51         'mann_whitney': mann_whitney_two_sample(pci_front, pci_back),
52         'ks': ks_two_sample(pci_front, pci_back),
53         'permutation': permutation_test_pci(scores, n_front=n_front),
54     }
55
56
57 # =====
58 # 第五部分: 推断性分析可视化
59 # =====
60
61 # --- 5.1 全特征检验相关图表 ---
62
63 def plot_top_words(feature_results):
64     """差异最显著词语的 -log10(p) 条形图。"""
65     words_data = feature_results['top_fdr_words']
66     if not words_data:
67         words_data = feature_results['top_records'][:15]
68
69     words = [d['word'] for d in words_data]
70     neg_log_p = [-np.log10(max(d['p'], 1e-300)) for d in words_data]
71
72     fig, ax = plt.subplots(figsize=(10, 6))
73     y_pos = np.arange(len(words))
74     ax.barh(y_pos, neg_log_p, color='steelblue')
75     ax.set_yticks(y_pos)
76     ax.set_yticklabels(words)
77     ax.invert_yaxis()
78     ax.set_xlabel(r'$-\log_{10}(p)$')
79     ax.set_title('差异最显著的前 15 个词语 (Mann-Whitney)')
80     ax.axvline(-np.log10(ALPHA), color='red', linestyle='--', linewidth=0.8, label=f'={ALPHA}')
81     ax.legend()
82     return _save_fig('top_diff_words.png')
83
84
85 def plot_feature_boxplots(features, representative, n_front=N_FRONT):
86     """代表词的归一化词频箱线图 (前 80 回 vs 后 40 回)。"""
87     n_rep = len(representative)
88     n_cols = min(3, n_rep)
89     n_rows = (n_rep + n_cols - 1) // n_cols
90     fig, axes = plt.subplots(n_rows, n_cols, figsize=(4 * n_cols, 4 * n_rows))
91     axes = np.atleast_1d(axes).flatten()
92
93     for ax, rec in zip(axes, representative):
94         idx = rec['index']
95         data_front = features[:n_front, idx]
96         data_back = features[n_front:, idx]
97         ax.boxplot([data_front, data_back], tick_labels=['前 80 回', '后 40 回'])
98         jitter_front = np.random.default_rng(0).uniform(-0.08, 0.08, len(data_front))
99         jitter_back = np.random.default_rng(1).uniform(-0.08, 0.08, len(data_back))
100         ax.scatter(1 + jitter_front, data_front, alpha=0.4, s=15, color='C0')
101         ax.scatter(2 + jitter_back, data_back, alpha=0.4, s=15, color='C1')
102         ax.set_title(f'[{rec["word"]}]\tp={rec["p"]:.2e}')
103         ax.set_ylabel('归一化词频')
104
105     for ax in axes[n_rep:]:
106         ax.set_visible(False)
107
108     fig.suptitle('代表性差异词语 — 分组箱线图')
109     return _save_fig('feature_boxplot.png')
110
111

```

```

112 def plot_feature_density(features, representative, n_front=N_FRONT):
113     """代表词的分布密度对比图（直方图 + KDE）。"""
114     n_rep = len(representative)
115     n_cols = min(3, n_rep)
116     n_rows = (n_rep + n_cols - 1) // n_cols
117     fig, axes = plt.subplots(n_rows, n_cols, figsize=(4 * n_cols, 4 * n_rows))
118     axes = np.atleast_1d(axes).flatten()
119
120     for ax, rec in zip(axes, representative):
121         idx = rec['index']
122         data_front = features[:n_front, idx]
123         data_back = features[n_front:, idx]
124         ax.hist(data_front, bins=15, density=True, alpha=0.5, label='前 80 回')
125         ax.hist(data_back, bins=15, density=True, alpha=0.5, label='后 40 回')
126         try:
127             x_front = np.linspace(data_front.min(), data_front.max(), 200)
128             x_back = np.linspace(data_back.min(), data_back.max(), 200)
129             ax.plot(x_front, stats.gaussian_kde(data_front)(x_front))
130             ax.plot(x_back, stats.gaussian_kde(data_back)(x_back))
131         except np.linalg.LinAlgError:
132             pass
133         ax.set_title(f'「{rec["word"]}」 KS p={rec["ks_p"]:.2e}')
134         ax.set_xlabel('归一化词频')
135         ax.legend(fontsize=8)
136
137     for ax in axes[n_rep:]:
138         ax.set_visible(False)
139
140     fig.suptitle('代表性差异词语 — 分布密度对比')
141     return _save_fig('feature_density.png')
142
143
144 def run_feature_visualization(features, feature_results):
145     """生成全特征检验相关的全部图表。"""
146     return [
147         plot_top_words(feature_results),
148         plot_feature_boxplots(features, feature_results['representative']),
149         plot_feature_density(features, feature_results['representative']),
150     ]
151
152
153 # --- 5.2 PCA 探索性可视化图表 ---
154
155 def plot_pca_scatter(scores, n_front=N_FRONT):
156     """PCA 三维散点图：前 80 回 vs 后 40 回。"""
157     fig = plt.figure(figsize=(9, 7))
158     ax = fig.add_subplot(111, projection='3d')
159     ax.scatter(
160         scores[:n_front, 0], scores[:n_front, 1], scores[:n_front, 2],
161         alpha=0.7, label='前 80 回', s=30,
162     )
163     ax.scatter(
164         scores[n_front:, 0], scores[n_front:, 1], scores[n_front:, 2],
165         alpha=0.7, label='后 40 回', s=30,
166     )
167     ax.set_xlabel('PC1')
168     ax.set_ylabel('PC2')
169     ax.set_zlabel('PC3')
170     ax.set_title('PCA 三维散点图：前 80 回 vs 后 40 回')
171     ax.legend()
172     return _save_fig('pca_scatter.png')
173
174
175 def plot_scee(explained_ratio_full):
176     """PCA 碎石图：展示各主成分方差解释率。"""
177     fig, ax = plt.subplots(figsize=(8, 4))
178     x = np.arange(1, len(explained_ratio_full) + 1)
179     ax.bar(x, explained_ratio_full * 100, alpha=0.7, label='单个主成分')
180     ax.plot(x, np.cumsum(explained_ratio_full) * 100, 'o-', color='C1', label='累计解释率')
181     ax.set_xlabel('主成分')
182     ax.set_ylabel('方差解释率 (%)')
183     ax.set_title('PCA 碎石图')
184     ax.legend()
185     return _save_fig('pca_scee.png')

```

```

186
187
188 def plot_pci_boxplot(scores, pci_results, n_front=N_FRONT):
189     """PC1 分组箱线图 (含散点抖动)。"""
190     mw_p = pci_results['mann_whitney']['p']
191     fig, ax = plt.subplots(figsize=(6, 4))
192     data_front = scores[:n_front, 0]
193     data_back = scores[n_front:, 0]
194     ax.boxplot([data_front, data_back], tick_labels=['前 80 回', '后 40 回'])
195     jitter_front = np.random.default_rng(0).uniform(-0.08, 0.08, len(data_front))
196     jitter_back = np.random.default_rng(1).uniform(-0.08, 0.08, len(data_back))
197     ax.scatter(1 + jitter_front, data_front, alpha=0.4, s=20, color='C0')
198     ax.scatter(2 + jitter_back, data_back, alpha=0.4, s=20, color='C1')
199     ax.set_title(f'PC1 分组箱线图 Mann-Whitney p={mw_p:.2e}')
200     ax.set_ylabel('PC1 得分')
201     return _save_fig('pci_boxplot.png')
202
203
204 def plot_pci_density(scores, pci_results, n_front=N_FRONT):
205     """PC1 分组分布密度对比图。"""
206     ks_p = pci_results['ks']['p']
207     fig, ax = plt.subplots(figsize=(6, 4))
208     data_front = scores[:n_front, 0]
209     data_back = scores[n_front:, 0]
210     ax.hist(data_front, bins=20, density=True, alpha=0.5, label='前 80 回')
211     ax.hist(data_back, bins=20, density=True, alpha=0.5, label='后 40 回')
212     try:
213         x_front = np.linspace(data_front.min(), data_front.max(), 200)
214         x_back = np.linspace(data_back.min(), data_back.max(), 200)
215         ax.plot(x_front, stats.gaussian_kde(data_front)(x_front), label='前 80 回 KDE')
216         ax.plot(x_back, stats.gaussian_kde(data_back)(x_back), label='后 40 回 KDE')
217     except np.linalg.LinAlgError:
218         pass
219     ax.set_title(f'PC1 分布对比 KS p={ks_p:.2e}')
220     ax.set_xlabel('PC1 得分')
221     ax.legend(fontsize=8)
222     return _save_fig('pci_density.png')
223
224
225 def run_pca_analysis(features):
226     """
227     PCA 降维、PC1 三项检验，并生成相关图表。
228     返回: scores, pca_ratio, pci_results, 图表路径列表。
229     """
230     scores, pca_ratio, explained_full = run_pca(features)
231     pci_results = pci_tests(scores)
232     paths = [
233         plot_pca_scatter(scores),
234         plot_scree(explained_full),
235         plot_pci_boxplot(scores, pci_results),
236         plot_pci_density(scores, pci_results),
237     ]
238     return scores, pca_ratio, pci_results, paths
239
240
241 # =====
242 # 第六部分: 统计报告
243 # =====
244 def format_report(desc_stats, feature_results, pca_ratio, pci_results, n_chapters, n_features,
245                  saved_paths, report_path=REPORT_PATH):
246     """组装完整文本报告 (描述性统计在前, 推断结果在后)。"""
247     sep = '=' * 60
248     pc_labels = ', '.join(f'PC{i + 1}' for i in range(len(pca_ratio)))
249     lines = []
250
251     def add(text=''):
252         lines.append(text)
253
254     add(format_descriptive_report(desc_stats))
255
256     # ---- 数据概况 ----
257     add(f'\n{sep}')
258     add('推断分析数据概况')
259     add(sep)

```

```

260 add(f' 章节数: {n_chapters}')
261 add(f' 特征维度 V: {n_features}')
262 add(f' 前 80 回样本量: {N_FRONT}')
263 add(f' 后 40 回样本量: {n_chapters - N_FRONT}')
264
265 # ----- 全特征检验 (核心推断, 优先呈现) -----
266 add(f'\n{sep}')
267 add(f'一、全特征逐词检验 ({n_features} 特征) ')
268 add(sep)
269 add(f' [Mann-Whitney U 检验 + 多重比较校正]')
270 add(f' 原始 p < {ALPHA} 的特征数: {feature_results["sig_raw"]}')
271 add(f' Bonferroni 校正后显著: {feature_results["sig_bonferroni"]}')
272 add(f' FDR (BH) 校正后显著: {feature_results["sig_fdr"]}')
273 add(f'\n Top 10 差异词 (按原始 p 值排序) :')
274 add(f' {"词语":<12} {"U":>10} {"p":>12} {"Bonf.p":>12} {"FDR.p":>12} {"r":>8}')
275 for rec in feature_results['top_records'][:10]:
276     add(f' {rec["word"]:<12} {rec["U"]:>10.1f} {rec["p"]:>12.4e} '
277         f'{rec["bonferroni_p"]:>12.4e} {rec["fdr_p"]:>12.4e} {rec["rank_biserial_r"]:>8.4f}')
278
279 add(f'\n 代表性特征补充检验 (Top {len(feature_results["representative"])}, Mann-Whitney + KS) :')
280 add(f' {"词语":<12} {"MW.p":>12} {"KS D":>8} {"KS p":>12} {"r":>8}')
281 for rec in feature_results['representative']:
282     add(f' {rec["word"]:<12} {rec["p"]:>12.4e} {rec["ks_D"]:>8.4f} '
283         f'{rec["ks_p"]:>12.4e} {rec["rank_biserial_r"]:>8.4f}')
284
285 # ----- PCA 降维 + PC1 补充检验 -----
286 add(f'\n{sep}')
287 add(f'二、PCA 降维与 PC1 补充检验')
288 add(sep)
289 add(f' 主成分数: {len(pca_ratio)}')
290 for i, ratio in enumerate(pca_ratio, 1):
291     add(f' PC{i} 解释方差比: {ratio:.4f} ({ratio * 100:.2f}%)')
292 add(f' {pca_labels} 累计解释率: {pca_ratio.sum():.4f} ({pca_ratio.sum() * 100:.2f}%)')
293 add(f' 说明: PC1 解释方差最大, 对其做 Mann-Whitney / KS / 置换检验作为 PCA 方向的补充验证。')
294
295 mw = pc1_results['mann_whitney']
296 ks = pc1_results['ks']
297 perm = pc1_results['permutation']
298 add(f'\n [PC1 — Mann-Whitney U 检验]')
299 add(f' U={mw["U"]:.1f}, p={mw["p"]:.4e}, rank-biserial r={mw["rank_biserial_r"]:.4f}')
300 add(f' [PC1 — Kolmogorov-Smirnov 两样本检验]')
301 add(f' D={ks["statistic"]:.4f}, p={ks["p"]:.4e}')
302 add(f' [PC1 — 置换检验 (两组 PC1 均值差的绝对值)]')
303 add(f' T_obs={perm["T_obs"]:.6f}, p_perm={perm["p"]:.4e}, 置换次数={perm["n_perm"]}')
304
305 # ----- 综合结论 -----
306 add(f'\n{sep}')
307 add(f'结论')
308 add(sep)
309 evidence = []
310 if feature_results['sig_raw'] > 0:
311     evidence.append(f'原始检验有 {feature_results["sig_raw"]} 个词语 p < {ALPHA}')
312 if feature_results['sig_bonferroni'] > 0:
313     evidence.append(f'Bonferroni 校正后有 {feature_results["sig_bonferroni"]} 个词语显著')
314 if feature_results['sig_fdr'] > 0:
315     evidence.append(f'FDR 校正后有 {feature_results["sig_fdr"]} 个词语显著差异')
316 rep_sig_ks = sum(1 for rec in feature_results['representative'] if rec['ks_p'] < ALPHA)
317 if rep_sig_ks > 0:
318     evidence.append(f'代表性特征中有 {rep_sig_ks} 个 KS 检验显著')
319 if mw['p'] < ALPHA:
320     evidence.append(f'PC1 Mann-Whitney 检验显著')
321 if ks['p'] < ALPHA:
322     evidence.append(f'PC1 KS 检验显著')
323 if perm['p'] < ALPHA:
324     evidence.append(f'PC1 置换检验显著')
325
326 if evidence:
327     add(f' 在 = {ALPHA} 下, 全特征非参数检验提示前 80 回与后 40 回存在用词差异: ')
328     for item in evidence:
329         add(f' - {item}')
330     add(f' 综合而言, 用词风格在前后两部分之间并非完全一致, 与「后四十回作者存疑」的假设相容。')
331 else:
332     add(f' 在 = {ALPHA} 下, 各项检验均未发现显著差异, 不能拒绝「前后回风格相同」的假设。')
333

```

```

334     add(f'\n{sep}')
335     add('图表已保存')
336     add(sep)
337     for path in saved_paths:
338         add(f' {path}')
339     add(f'\n{sep}')
340     add('检验报告已保存')
341     add(sep)
342     add(f' {report_path}')
343
344     return '\n'.join(lines)
345
346
347 def save_report(report_text, report_path=REPORT_PATH):
348     """将报告写入 txt 文件。"""
349     with open(report_path, 'w', encoding='utf-8') as f:
350         f.write(report_text)
351     return report_path
352
353
354 def print_report(desc_stats, feature_results, pca_ratio, pci_results, n_chapters, n_features,
355                 saved_paths, report_path=REPORT_PATH):
356     """打印并保存统计报告。"""
357     report_text = format_report(
358         desc_stats, feature_results, pca_ratio, pci_results,
359         n_chapters, n_features, saved_paths, report_path=report_path,
360     )
361     print(report_text, flush=True)
362     save_report(report_text, report_path=report_path)
363
364
365 # =====
366 # 主流程
367 # =====
368 def main():
369     os.makedirs(OUTPUT_DIR, exist_ok=True)
370
371     # ---- 步骤 0: 预处理, 构建特征矩阵 ----
372     print('【步骤 0】文本预处理与特征构建...')
373     features, word_list, chapter_ids, preprocess_meta = prepare_features()
374
375     # ---- 步骤 1: 描述性统计与预处理可视化 ----
376     print('【步骤 1】描述性统计与预处理可视化...')
377     desc_stats = compute_descriptive_stats(preprocess_meta, features, word_list, chapter_ids)
378     desc_plot_paths = run_preprocessing_visualization(desc_stats, features, word_list)
379     print(format_descriptive_report(desc_stats))
380
381     # ---- 步骤 2: 全特征非参数检验 (核心推断) ----
382     print('【步骤 2】全特征 Mann-Whitney 检验 + 多重比较校正...')
383     feature_results = feature_wise_tests(features, word_list)
384
385     print('【步骤 2】生成检验相关图表...')
386     feature_plot_paths = run_feature_visualization(features, feature_results)
387
388     # ---- 步骤 3: PCA 降维 + PC1 补充检验 ----
389     print('【步骤 3】PCA 降维、PC1 三项检验与可视化...')
390     _, pca_ratio, pci_results, pca_plot_paths = run_pca_analysis(features)
391
392     saved_paths = desc_plot_paths + feature_plot_paths + pca_plot_paths
393
394     # ---- 步骤 4: 输出报告 ----
395     print('【步骤 4】输出统计报告...')
396     print_report(
397         desc_stats, feature_results, pca_ratio, pci_results,
398         n_chapters=len(chapter_ids), n_features=len(word_list), saved_paths=saved_paths,
399     )
400
401     if SHOW_PLOTS:
402         plt.show()

```